
MUHAMMAD TAHA

22F-3277

LAB 11

03/11/2023

LAB 11

(Total Marks 15+3=18)

Task 1: Write the program that count the number of spaces in given string using SCAS instruction and display the counter on screen

- Determine the length of a string using the null terminator.
- Only use **SCAS instruction**
- Message: db 'Ass em bly lan gua ge', 0 counter: 5
- Use your full name in message and put space after two alphabets.

org 0x0100

jmp start

message: db 'Mu ha mm ad Ta ha', 0

clrscr:

push es
push ax
push cx
push di

mov ax,0xb800
mov es,ax
mov cx,2000
mov ax,0x0720

cld
rep stosw

pop di
pop cx
pop ax
pop es

ret

countspaces:

push bp
mov bp,sp
push ax
push es
push cx
push di

;;;;;;;;

;calculating length

les di, [bp+4] ;es:di to string, es point to segment, di point to string
mov cx, 0xffff
xor al,al
repne scasb

mov ax, 0xffff
sub ax, cx
dec ax ;minus null value from string length
mov cx, ax ;moves length of string in ax

;;;;;;;;

les di, [bp+4] ;es:di is again set correct
mov al, ''
mov bx, 0 ;bx counts spaces

next:

repne scasb
jne printCount
inc bx
jmp next

call printCount

pop di
pop cx
pop es
pop ax
pop bp
ret 4

printCount:

push bx
push ax
push di
push es

```
mov ax, 0xb800
mov es, ax
mov di, 160

add bl, 0x30
mov bh, 4h          ;red color
mov ax, bx
add di, 2
mov [es:di], ax ; print char on screen

pop di
pop di
pop ax
pop bx
ret
```

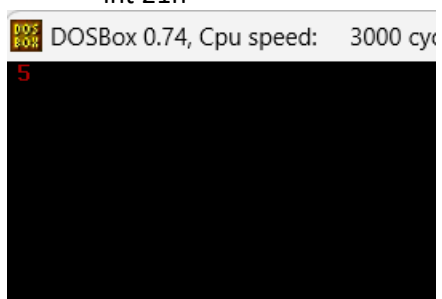
start:

```
call clrscr
```

```
push ds
mov ax, message
push ax
```

```
call countspaces
```

```
mov ax, 0x4c00
int 21h
```



Task 2: Write a code for scroll down behavior of the screen. This will require an input of how many numbers of lines being needed to be scrolled up as a parameter and then you will use “Mvs” to scroll down.

```
; scroll down the screen
[org 0x0100]
jmp start
; subroutine to scrolls down the screen
; take the number of lines to scroll as parameter
scrollDown:
    push bp
    mov bp,sp
    push ax
    push cx
    push si
    push di
    push es
    push ds

    mov ax, 80 ; load chars per row in ax
    mul byte [bp+4] ; calculate source position
    push ax ; save position for later use
    shl ax, 1 ; convert to byte offset
    mov si, 3998 ; last location on the screen
    sub si, ax ; load source position in si

    shr ax,1      ;;div ax by 2 so now we know the number of words

    mov cx, 2000 ; number of screen locations
    sub cx, ax ; count of words to move
    mov ax, 0xb800
    mov es, ax ; point es to video base
    mov ds, ax ; point ds to video base
    mov di, 3998 ; point di to lower right column

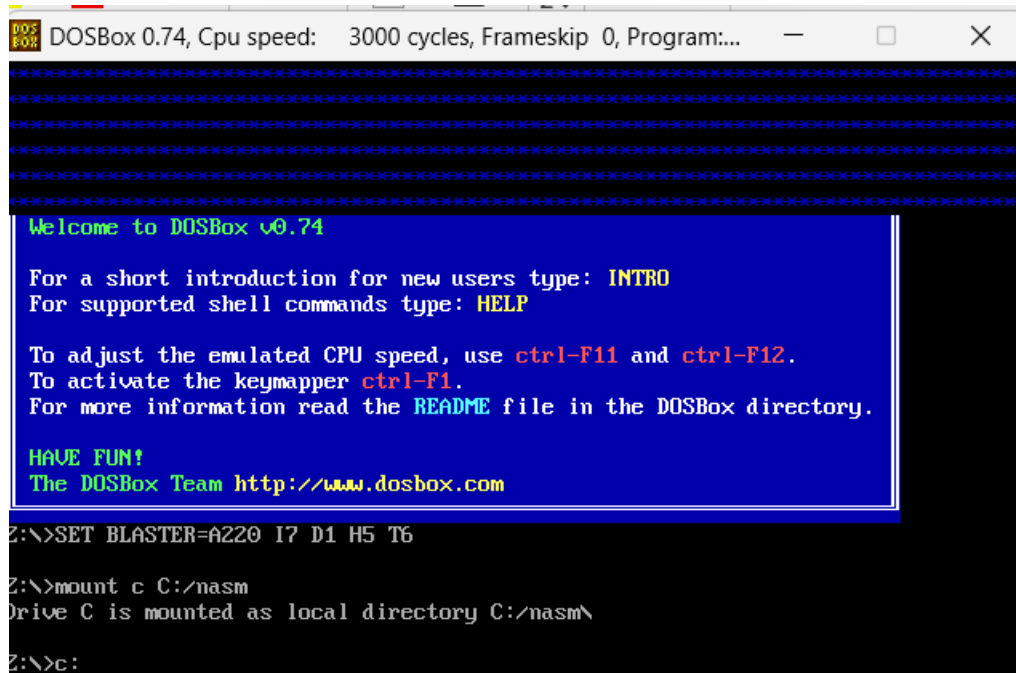
    std ; set auto decrement mode
    rep movsw ; scroll up
    mov ax, 0x12a ;;put star above
    pop cx ; count of positions to clear
    rep stosw ; clear the scrolled space

pop ds
pop es
pop di
pop si
```

```
pop cx
pop ax
pop bp
ret 2
```

start:

```
mov ax,6
push ax ; push number of lines to scroll
call scrolldown ; call scroll down subroutine
mov ax, 0x4c00 ; terminate program
int 0x21
```



The image shows a DOSBox 0.74 window. The title bar reads "DOSBox 0.74, Cpu speed: 3000 cycles, Frameskip 0, Program:...". The main window has a black background with a blue rectangular box containing white text. The text inside the box is a welcome message for DOSBox v0.74, including instructions on how to use the program (e.g., using Ctrl-F11 for CPU speed, Ctrl-F12 for frameskip, and Ctrl-F1 for the keymapper) and a link to the DOSBox website. Below the box, the command prompt shows the user entering the command "SET BLASTER=A220 I7 D1 H5 T6". The prompt then shows the user entering "mount c C:/nasm", and the system responds with "Drive C is mounted as local directory C:/nasm\". Finally, the prompt shows the user entering "c:", which is the root of the C: drive.

```
DOSBox 0.74, Cpu speed: 3000 cycles, Frameskip 0, Program:...
Welcome to DOSBox v0.74

For a short introduction for new users type: INTRO
For supported shell commands type: HELP

To adjust the emulated CPU speed, use ctrl-F11 and ctrl-F12.
To activate the keymapper ctrl-F1.
For more information read the README file in the DOSBox directory.

HAVE FUN!
The DOSBox Team http://www.dosbox.com

Z:\>SET BLASTER=A220 I7 D1 H5 T6

Z:\>mount c C:/nasm
Drive C is mounted as local directory C:/nasm\

Z:\>c:
```

Task3: Write a program for substring search (that searches for a substring within a given string) The program should return the 1 if the substring is found, or 0 if the substring is not found.

- Use Cmps instruction
- Use ax register for returning value
- i.e message string: db 'hello world'
- search: db 'world'
- Use your full name in message string and once find first name in message string and then search wrong name.

```
; find substring from string
[org 0x0100]
jmp start
string: db 'Muhammad Taha', 0
substring1: db 'Taha', 0
substring2: db 'xyz', 0
```

clrscr:

```
push es
push ax
push cx
push di
```

```
mov ax,0xb800
mov es,ax
mov cx,2000
mov ax,0x0720
```

```
cld
rep stosw
```

```
pop di
pop cx
pop ax
pop es
ret
```

strlen:

```
push bp
push es
push cx
push di
push si
```

```
mov cx, 0xffff ; load maximum number in cx
xor al, al ; load a zero in al
repne scasb ; find zero in the string
mov ax, 0xffff ; load maximum number in ax
sub ax, cx ; find change in cx
dec ax ; exclude null from length
```

```
pop si
pop di
pop cx
pop es
pop bp
ret
```

findSubstring:

```
push bp
mov bp, sp
push cx
push si
push di
push es
```

```
mov es, [bp+8] ;points to ds
```

```
mov di, [bp+4] ;now es:di points to substring to calculate length
call strlen ;returns length of substring
mov bx, ax ;length of substring
```

```
mov di, [bp+6] ;point es:di to string
call strlen ;returns length of string
mov cx, ax ;length of string
```

```
mov si, [bp+4] ;point ds:si to substring
```

```
cmp bx, cx ;if substring is greater
jg notfound
```

```
;es:di points to string
;ds:si points to substring to find in string
```

l1:

```
repe cmpsb
cmp cx, 0
je exit
mov si, [bp+4]
jne l1
```

exit:

add bx, [bp+4] ;adding substring index to the length of substring and then comparing if it is equal to current si value then it means that the substring was found

```
cmp si, bx
je found
jne notfound
```

```
found:
mov ax, 1
jmp return
```

```
notfound:
mov ax, 0
```

return:

```
pop es
pop di
pop si
pop cx
pop bp
ret 6
```

printAX:

```
push di
push ax
push bx
push es
```

```
mov bx, 0xb800
mov es, bx
```

```
add al, 0x30
mov ah, 02
mov word [es:di], ax
```

```
pop es
pop bx
pop ax
pop di
ret 4
```

start:

```
call clrscr
```



```
push ds
mov ax, string
push ax
mov ax, substring1
push ax
```

```
call findSubstring
```

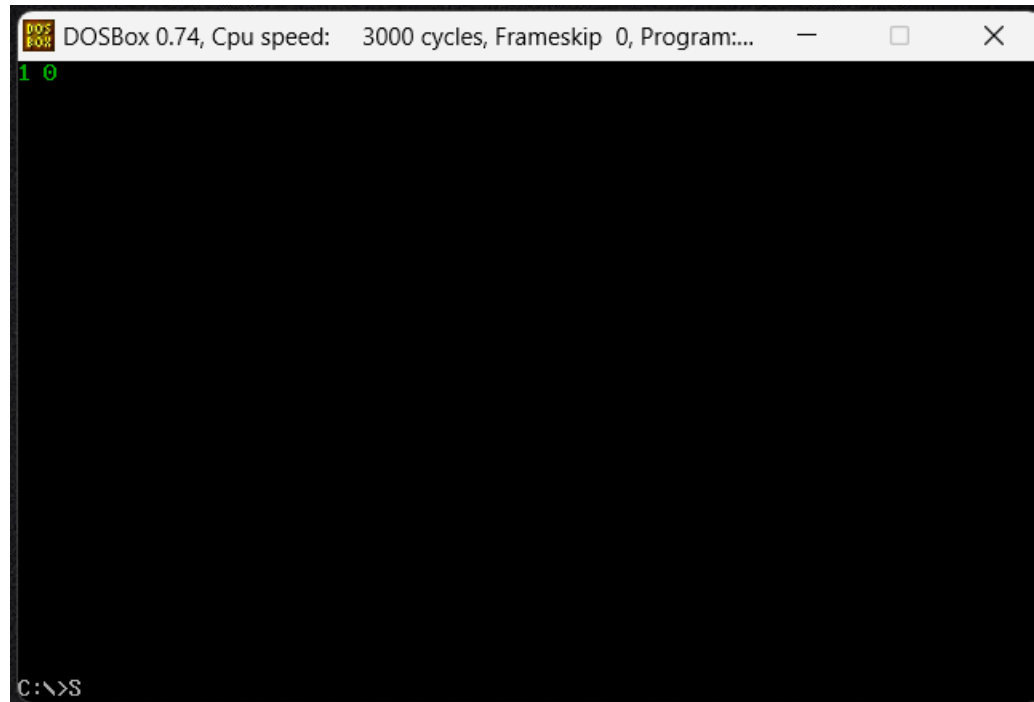
```
push ax
mov di, 160      ;push di value
push di
call printAX
```

```
push ds
mov ax, string
push ax
mov ax, substring2
push ax
```

```
call findSubstring ;returns 1 or 0 in AX
```

```
push ax
mov di, 164      ;push di value
push di
call printAX
```

```
mov ax, 0x4c00
int 0x21
```



Best of luck